

A Lightweight, Efficient and Explainable-by-Design Convolutional Neural Network for Internet Traffic Classification

Kevin Fauvel
Huawei Technologies Co., Ltd
France

Fuxing Chen
Huawei Technologies Co., Ltd
France

Dario Rossi
Huawei Technologies Co., Ltd
France

ABSTRACT

Traffic classification, i.e., the identification of the type of applications flowing in a network, is a strategic task for numerous activities (e.g., intrusion detection, routing). This task faces some critical challenges that current deep learning approaches do not address. The design of current approaches do not take into consideration the fact that networking hardware (e.g., routers) often runs with limited computational resources. Further, they do not meet the need for faithful explainability highlighted by regulatory bodies. Finally, these traffic classifiers are evaluated on small datasets which fail to reflect the diversity of applications in real-world settings.

Therefore, this paper introduces a new Lightweight, Efficient and eXplainable-by-design convolutional neural network (LEXNet) for Internet traffic classification, which relies on a new residual block (for lightweight and efficiency purposes) and prototype layer (for explainability). Based on a commercial-grade dataset, our evaluation shows that LEXNet succeeds to maintain the same accuracy as the best performing state-of-the-art neural network, while providing the additional features previously mentioned. Moreover, we illustrate the explainability feature of our approach, which stems from the communication of detected application prototypes to the end-user, and we highlight the faithfulness of LEXNet explanations through a comparison with post hoc methods.

CCS CONCEPTS

• Computing methodologies → Machine learning.

KEYWORDS

Deep Learning, Explainable AI, Internet Traffic Classification

1 INTRODUCTION

Traffic classification, i.e., the identification of the type of applications flowing in a network [16], is strategic for network operators to perform a wide range of network management activities [4, 8]. These activities include capacity planning, intrusion detection and service differentiation (e.g., for business critical applications).

The sharp rise of encrypted traffic in the last decade hampered traditional rule-based techniques, pushing the adoption of machine learning-assisted classification [42] to supplant deep packet inspection [13, 30, 41, 50]. Inspired by seminal work [15], a recent wave [2, 4, 7, 35, 36, 40, 48, 59] of deep learning models (Convolutional Neural Networks - CNNs, or recurrent architectures like Long Short-Term Memory - LSTMs) successfully tackles the encryption challenge by just leveraging the size and direction of the first few packets in a flow.

However, these studies still face major challenges. First, due to the widespread use of mobile devices and the vast diversity of mobile applications, *large-scale* encrypted traffic classification becomes increasingly difficult [3, 4]. At the same time, academic models are generally evaluated with a few tens of classes, which are significantly below commercial settings (hundreds to thousands classes) [13, 30, 50]. Second, even though focusing on relatively small datasets, academic models may still use a disproportionate amount of resources - models with millions of weights are commonplace as highlighted in [60]. This clashes with the need for near real-time classification (e.g., due to latency sensitive traffic) on the one hand, and the *limited computational resources* [18] available on network devices (e.g., routers) on the other, which renders a lightweight and efficient model necessary. Indeed, fast inference (~10k classifications/second) on traffic classification engines having limited computational budget (e.g., ARM CPUs, microcontrollers) requires model to be small in order to experience fewer computation during forward propagation. Last but not least, regulatory and standardization bodies highlight *faithful explainability* as a pillar for accountability, responsibility, and transparency of processes including AI components [19, 43, 45], which could prevent the deployment of the latest machine learning techniques that do not have this feature. Faithfulness is critical as it corresponds to the level of trust an end-user can have in the explanations of model predictions, i.e., the level of relatedness of the explanations to what the model actually computes [24]. In the context of traffic classification, a faithfully explainable model (explainable-by-design) would be beneficial from both a compliance and a business standpoint (e.g., SLAs). Nonetheless, the faithfulness of such a model should not negatively impact the prediction performance, nor come at the cost of an increased model computational and memory complexity. A few models in traffic classification [7, 40, 48] account for explainability, but they cannot provide faithful explainability as they rely on post hoc model-agnostic explainability [51]. With the above constraints in mind, we therefore propose a new Lightweight, Efficient and eXplainable-by-design CNN (LEXNet) for traffic classification, and evaluate it on a large-scale commercial-grade dataset.

An additional requirement that shapes the overall architecture of our classifier is the form of explanations. Based on network experts input, we have adopted explanations under the form of class-specific prototypes as they can be associated with application signatures and provided for each sample/flow to the end-user. Thus, as illustrated in Figure 1, the overall architecture of our LEXNet follows the typical 3-part structure of explainable-by-design prototype-based networks [10]: (i) a CNN backbone extracts discriminative features, then (ii) a prototype block computes similarities to the learned class-specific prototypes, and finally (iii) the classification is performed based on these similarity scores. Then, the contributions of this paper are the following:

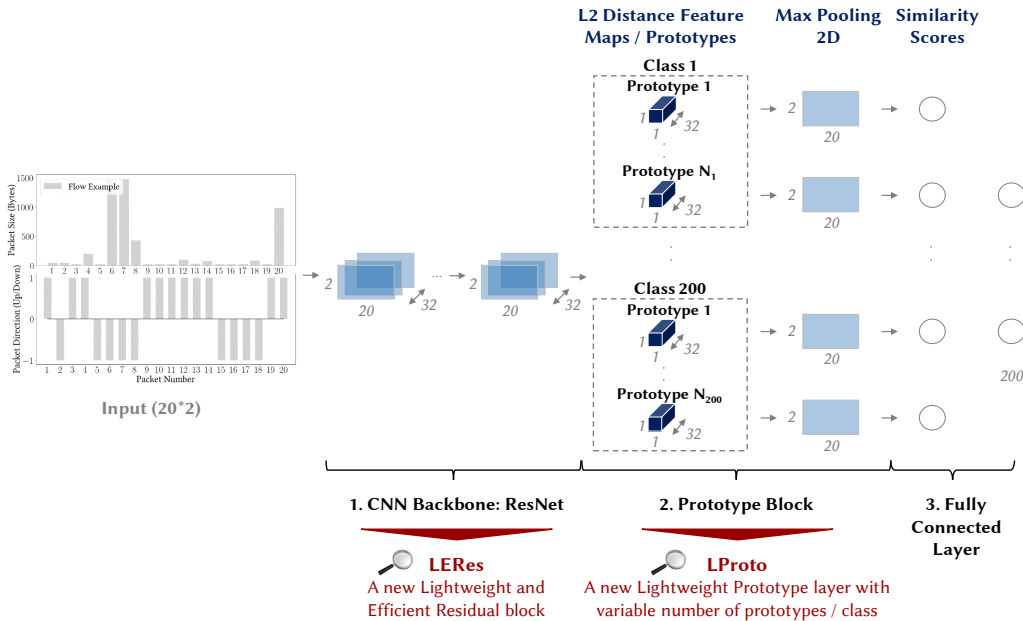


Figure 1: Illustration of LEXNet 3-part structure, on our commercial-grade dataset composed of 200 classes, with its two main contributions in red: LERes block and LProto layer. LERes is detailed in Figure 2 and LProto in Figure 3. N - Number.

- (i) *CNN backbone: LERes block* - LEXNet redesigns the widely used residual block [27] with cost-efficient operations (linear transformations and concatenate operations) which significantly reduce its number of parameters (-19%) and CPU inference time (-41%), while preserving the accuracy of the network (-0.7%). We called this new block LERes block;
- (ii) *Prototype block: LProto* - LEXNet converts the current state-of-the-art prototype block [10] into a unique prototype layer (called LProto). Preserving the explainability-by-design, LProto significantly reduces the number of parameters (-36%) and CPU inference time (-24%), while increasing the accuracy by 4%. In addition, in contrast with the prototype block which uses a unique number of prototypes per class as hyperparameter - same number for all classes, LProto automatically learns a variable number of prototypes per class during training to better characterize the discriminative features of each application;
- *Evaluation: Performance* - The evaluation on our commercial-grade dataset shows that LEXNet is more accurate than the current state-of-the-art explainable-by-design CNN (ProtoPNet [10]), and maintains the same accuracy as the best performing deep learning approach (ResNet [27]) that does not provide faithful explanations. Moreover, we demonstrate that LEXNet significantly reduces the model size and inference time compared to the state-of-the-art neural networks with explainability-by-design and post hoc explainability methods. Both our dataset [17] and code [14] are made publicly available to allow reproducibility. In addition, we evaluate LEXNet on current public benchmark datasets from different domains and show that the results are consistent with the ones obtained on our dataset;

- *Evaluation: Explainability* - Following the illustration of the explainability of our approach which stems from the communication of detected application/class prototypes to the end-user, we highlight the faithfulness of LEXNet explanations by comparing them to the ones provided by the state-of-the-art post hoc explainability methods.

2 RELATED WORK

Traffic classification can be formulated as a Multivariate Time Series (MTS) classification task, where the input is packet-level data (typically packet size and direction) related to a single flow¹, and the output is an application label. An example of a flow is represented in grey in Figure 1, with the packet size and direction (y-axis) of the first 20 packets (x-axis) of a flow belonging to Application 1. This paper addresses encrypted traffic classification as a MTS problem, when it has been tackled as an univariate one in the literature ($\pm$ packet size as input time series). Our multivariate approach offers more granularity and information in its explanations, with both per-packet size and direction as explanatory variables, without degradation in performance.

The state-of-the-art approaches [2, 7, 35, 36, 40, 48, 59] adopt heavyweight deep learning classifiers (from 1M to 2M weights), and do not discuss the impact of their model size on the accuracy and inference time. When accounting for explainability, these studies employ post hoc explainability methods like SHAP [37], which cannot meet the faithfulness requirement. Finally, the results of these studies are not directly comparable as they are evaluated on different datasets. Plus, these datasets (mostly private, from 8 to 80

¹A packet is a unit of data routed on the network, and a flow is a set of packets that shares the same 5-tuple (i.e., source IP, source port, destination IP, destination port, and transport-layer protocol) [58].

applications and from 8k to 950k flows) fail to reflect the diversity of applications in real commercial settings.

Consequently, an accurate, lightweight, efficient and explainable-by-design approach for traffic classification evaluated on a public commercial-grade dataset is necessary. We identified CNNs as having the potential to fulfill these needs. Despite the recent promising results of large multimodal models and their potential for addressing various network management tasks, we do not consider them in this study as these large models are not compatible with the limited computational hardware resources available in networks today, nor are they capable of integrating explainability-by-design for the extraction of faithful explanations to support their predictions. In the next sections, we present the corresponding machine learning state-of-the-art (MTS classification, explainability, lightweight and efficient architectures) on which we position our approach.

2.1 MTS Classification

The state-of-the-art MTS classifiers are composed of three categories: similarity-based [55], feature-based [6, 33, 53, 57] and deep learning methods [23, 32, 64]. The results published show that the top two most accurate MTS classifiers on average on the public UEA archive [5] are deep learning methods (top-1 XCM [23], top-2 MLSTM-FCN [32]). XCM extracts discriminative features related to observed variables and time directly from the input data using 2D and 1D convolutions filters, while MLSTM-FCN stacks a LSTM layer and a 1D CNN layer along with squeeze-and-excitation blocks to generate latent features. Therefore, in this work, we choose to benchmark LEXNet to these two MTS classifiers. Plus, we integrate in our benchmark the commonly adopted state-of-the-art CNNs in computer vision (ResNet [27], DenseNet [29]), the vanilla 1D CNN widely used in traffic classification studies, and the traditional machine learning methods Random Forest [9] and XGBoost [11].

2.2 Explainability

There are several methods belonging to two main categories [20]: explainability-by-design and post hoc explainability.

Post hoc methods are the most popular ones for deep learning models (model-specific methods as the saliency method Grad-CAM [54], or model-agnostic as the surrogate model SHAP [37]). However, some recent studies [12, 51] show that post hoc explainability methods face a faithfulness issue.

Lately, there have been some work proposing explainable-by-design CNN approaches [10, 12, 21, 63] to circumvent this issue. Two of these approaches [21, 63] provide the explainability-by-design feature but at the cost of some lag behind the state-of-the-art CNNs in terms of accuracy, which remains a prerequisite for our task. Then, Chen et al. (2020) present Concept Whitening, an alternative to a batch normalization layer which decorrelates the latent space and aligns its axes with known concepts of interest. This approach requires that the concepts/applications are completely decorrelated, so it is not suited for our traffic classification task as some applications can be correlated (e.g., different applications from the same editor). Finally, another approach combines accuracy with explainability-by-design: ProtoPNet [10]. It consists of a CNN backbone to extract discriminative features, then a prototype block that computes similarities to the learned class-specific prototypes

as basis for classification. This approach is particularly interesting for our task as it is aligned with the way humans describe their own thinking in classification, by focusing on parts of the input data and comparing them with prototypical parts of the data from a given class. In particular, prototypes can be used to form application signatures for network experts. Therefore, with the objective to combine accuracy and explainability-by-design suited for traffic classification, we adopted a prototype-based CNN approach. Then, we worked to reduce the number of weights and inference time of the approach while maintaining the accuracy, in particular with regard to the CNN backbone.

2.3 Lightweight and Efficient Architectures

Multiple studies about lightweight and efficient CNN architectures have been published over the last years: ShuffleNetV2 [39] and its four practical guidelines, EfficientNet [56] with its scaling method along depth/width/resolution dimensions, MobileNetV3 [28] with its network search techniques and optimized nonlinearities, GhostNet [26] using a Ghost module based on linear transformations, and CondenseNetV2 [61] relying on a Sparse Feature Reactivation module.

These CNNs are evaluated on the same public dataset (ImageNet). However, most of these evaluations optimize a proxy metric to measure efficiency (number of floating-point operations per second - FLOPs), which is not always equivalent to the direct metric we care about - the inference time [39]. Using FLOPs as the reference, these studies do not evaluate the accuracy and inference time on a comparable model size (number of weights). A recent work [49] shows that ResNet architecture is usually faster than its latest competitors, offering a better accuracy/inference time trade-off (which is also confirmed by our experiments in Section 5.1). Therefore, we adopted ResNet as CNN backbone and revisited its residual block to make it lighter and more efficient. We include ShuffleNetV2, EfficientNet, MobileNetV3, GhostNet and CondenseNetV2 in our benchmark, and perform a comparison of the accuracy and inference time based on a comparable model size in Section 5.1.

3 ALGORITHM

We now propose our new Lightweight, Efficient and eXplainable-by-design CNN classifier (LEXNet). The overall architecture of LEXNet follows the one of explainable-by-design prototype-based networks like the state-of-the-art ProtoPNet[10]. Before introducing our contributions, we present the structure and functioning of a prototype-based network. As illustrated in Figure 1, the network consists of three parts. First, it is composed of a CNN backbone for feature extraction (see 1. in Figure 1). In the context of LEXNet, as shown in section 5.1, ResNet [27] has been selected as it obtains the best accuracy. Then, a prototype block learns class-specific prototypes, with the size of the prototypes set as hyperparameter of the algorithm. Prototypes of size (1, 1) are chosen (see 2. in Figure 1) to be able to precisely identify discriminative features and provide valuable explanations to the end-user. Each prototype in the block computes the L2 distances with all patches of the last convolutional layer (e.g., size of the last layer in Figure 1: $20 \times 2 \times 32$), and inverts the distances into similarity scores. The result is an activation map of similarity scores (e.g., size of a similarity matrix in Figure 1:

20×2). This activation map preserves the spatial relation of the convolutional output, and can be upsampled to the size of the input sample to produce a heatmap that identifies the part of the input sample most similar to the learned prototype. To make sure that the learned class-specific prototypes correspond to training samples patches, the prototypes are projected onto the nearest latent training patch from the same class during the training of the model. The activation map of similarity scores produced by each prototype is then reduced using global max pooling to a single similarity score, which indicates how strong the presence of a prototypical part is in some patch of the input sample. Finally, the classification is performed with a fully connected layer based on these similarity scores (see 3. in Figure 1).

Then, LEXNet has two main contributions. Our first contribution is to introduce a new lightweight and efficient residual block which reduces the number of weights and inference time of the original residual block, while preserving the accuracy of ResNet. To further reduce the size of LEXNet and improve its accuracy, we propose as a second contribution to replace the prototype block with a new lightweight prototype layer. This new prototype layer also allows a better characterization of each class by automatically learning a variable number of prototypes per class. We present in the next sections these two contributions, and end with the overall architecture of our network and its training procedure.

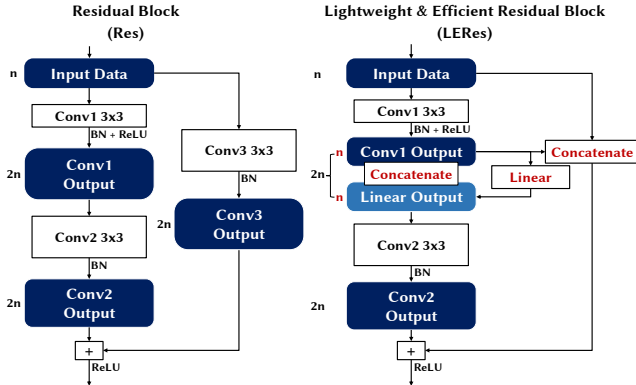


Figure 2: ResNet original residual block (Res) and our new Lightweight and Efficient Residual block (LERes) with its contributions in red.

3.1 Lightweight and Efficient Residual Block

As previously introduced, we select ResNet [27] for the CNN backbone as it obtains the best accuracy on our traffic classification dataset (detailed in Section 5.1). ResNet is a state-of-the-art CNN composed of consecutive residual blocks which consist of two convolutions with a shortcut added to link the output of a block to its input in order to reduce the vanishing gradient effect [62]. When the number of output channels equals the one of the input channels, the input data is directly added to the output of the block with the shortcut. Nonetheless, the prediction performance of ResNet is supported by an increase in the number of channels along the depth of the network to learn elaborated features, which implies residual

blocks with a higher number (usually twice) of output channels than input channels. In this case, before performing the addition, another convolution is applied to the input data in order to match the dimensions of the block output. A residual block is illustrated on the left side of Figure 2 with n input channels and $2n$ output channels. We can see in this Figure that two convolutions have a different number of output channels than input channels (Convolutions 1 and 3). However, a guideline from [39] (ShuffleNetV2) states that efficient convolutions should keep an equal channel width to minimize memory access cost and inference time. Therefore, we propose a new residual block that only performs convolutions with an equal channel width, and the additional feature maps are generated with two cheap operations (see right side of Figure 2).

First, the authors of GhostNet [26] state that CNNs with good prediction performance have redundancy in feature maps, and that substituting some of the feature maps with a linear transformation of the same convolution output does not impact the prediction performance of the network. Therefore, we double the number of feature maps from the first convolution in a cost-efficient way using a series of linear transformations of its output (3×3 linear kernel). These new feature maps are concatenated to the ones from the first convolution output to form the input of the second convolution². Thus, the first and second convolutions in our new residual block have an equal channel width (n and $2n$ respectively, see Figure 2).

The second operation concerns the shortcut link: instead of converting the input data to the block output dimensions with a third convolution (see Res block in Figure 2), we propose to save this convolution by keeping the input data and concatenating it with the output from the first convolution. Our experiments on the traffic classification dataset show that the new residual block LERes allows ResNet to reduce the number of weights of the backbone by 19.1% and the CPU inference time by 41.3% compared to ResNet with the original residual block, while maintaining its accuracy (99.3% of the original accuracy).

3.2 Lightweight Prototype Layer

Next, we adopt a new lightweight prototype layer instead of the prototype block of ProtoPNet. As illustrated in Figure 3, the prototype block of ProtoPNet adds two convolutional layers with an important number of channels to the CNN backbone (recommended setting: ≥ 128). Then, it connects the prototype layer with a Sigmoid activation function. Finally, the prototypes have the same depth as the last convolutional layer (e.g., 128) for the computation of the similarity matrices with the L2 distance. We propose to remove the two additional convolutional layers (see right block of Figure 3). As a consequence, the prototypes have a much smaller depth (same as CNN backbone output: 32 versus 128). This allows us to reduce the number of weights by 29.4% compared to the original prototype block of ProtoPNet, and implies a 17.0% reduction in CPU inference time. Then, we replace the last activation function from the CNN backbone with a Sigmoid. As suggested in [52] and confirmed by our experiments (see Section 5.1), replacing the ReLU activation

²We have also considered having both convolutions 1 and 2 with an equal channel width n , and moving the concatenate operation with the feature maps from the linear transformation after the convolution 2. However, our experiments showed that this configuration leads to a decrease in accuracy without noticeable drop in inference time.

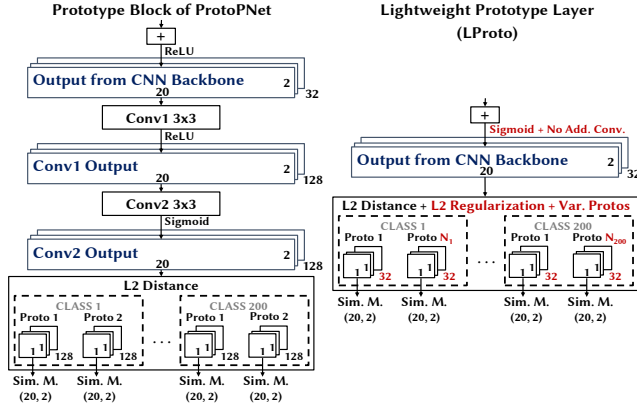


Figure 3: The prototype block of ProtoPNet and the new Lightweight Prototype Layer of LEXNet (LProto) with its contributions in red. Sim. M. - similarity matrix.

function from the last layer of our CNN backbone with a Sigmoid improves the accuracy of our network. We further improve the accuracy of our network by adding an L2 regularization on the weights of the prototypes in order to enhance its generalization ability. Considering the limited number of prototypes, and supported by the results from our experiments, we have selected an L2 regularization over an L1 sparse solution. Finally, LEXNet learns a variable number of prototypes per class ($N_1, \dots, N_{200}$ - see Figure 3), while the existing prototype block in ProtoPNet requires a fixed number of prototypes per class, set as hyperparameter of the model. This new feature allows LEXNet to characterize each class with an adequate number of prototypes and better inform the end-user about the discriminative part among classes, while reducing the total number of prototypes (340 versus 400). Overall, our new prototype layer reduces the number of weights by 36% and CPU inference time by 24% compared to the original prototype block, while increasing its accuracy by 4% - split in Section 5.1 The next section details the learning procedure of LEXNet.

3.3 Network Training

We present LEXNet pseudocode in Algorithm 1. Its training procedure is composed of three stages. First, as other prototype networks [10], the procedure updates the weights of the backbone and prototypes while keeping the weights of the last layer constant (see Stage 1.). Next, following a predetermined number of epochs (hyperparameter N_{SGD} - 20 in our setting), the procedure updates the prototypes by projecting them onto the nearest latent training patch from the same class (see Stage 2.). Then, in order to learn a variable number of prototypes per class (1 prototype per class at the initialization of the network to minimize the size of the model), we compute the average distance of all samples to the closest prototype of their class to evaluate how discriminative the learned prototypes are across the dataset (size of avg_dists : $1 \times N_{classes}$). It is desirable to have low distances on average for all classes, which would imply that the prototypes are well representing their class. Therefore, we adopt the Kurtosis to assess the distribution of distances, i.e., a measure of the tailedness of a distribution. If the distribution is

fat-tailed (kurtosis > 0 - kurtosis of the normal distribution), we add one prototype for each class in the 25-th percentile of the distribution in order to better characterize the diversity of samples in that classes. Finally, we update the weight of the last layer only while keeping the weights of the backbone and prototypes constant for a predetermined number of epochs (hyperparameter N_{last} - see Stage 3.). These three stages are repeated until reaching the global number of epochs (N_{epochs}).

Algorithm 1 LEXNet

```

1: Initialize:  $w_{back} \leftarrow$  Kaiming uniform initialization;  $\forall j$ : prototype  $p_j \leftarrow$ 
   Uniform( $[0, 1]^{H \times W \times D}$ );  $\forall k, j$ :  $w_h^{(k,j)} \leftarrow 1$  if  $p_j \in P_k$ ,
    $w_h^{(k,j)} \leftarrow -0.5$  if  $p_j \notin P_k$ 
2: for epoch  $t = 1, \dots, N_{epochs}$  do
3:   /* Stage 1: SGD of layers before the last */
4:   for SGD training epoch  $t' = t + 1, \dots, t + N_{SGD}$  do
5:     if  $t' < 5$  then /* Warm-up of the backbone */
6:        $w_{back} \leftarrow$  update weights of the backbone;
7:        $w_{back} \leftarrow$  update weights of the backbone;
8:        $P \leftarrow$  update weights of the prototypes;
9:        $t \leftarrow t + N_{SGD}$ 
10:    /* Stage 2: update of the prototypes */
11:    for each prototype  $p_j$  do
12:       $p_j \leftarrow$  project prototype onto the nearest latent training patch from the same class;
13:       $dists \leftarrow$  min distance of each sample to the prototypes of its class;
14:       $avg\_dists \leftarrow$  average  $dist$  per class across all samples;
15:      if kurtosis( $avg\_dists$ )  $> 0$  then
16:         $classes \leftarrow$  classes in 25-th percentile of  $avg\_dists$ ;
17:        for each  $k$  in  $classes$  do
18:           $P_k \leftarrow$  add 1 prototype;
19:      /* Stage 3: training of the last layer */
20:      for training epoch  $t'' = 1, \dots, N_{last}$  do
21:         $w_h \leftarrow$  update weights of the last layer

```

3.4 Network Architecture

The overall architecture of LEXNet is presented in Table 1. In order to limit the size of our network, the number of LERes blocks has been determined by cross-validation on the training set. Thus, additional LERes blocks would not increase the accuracy of the network on our traffic classification dataset. A higher number of filters would not increase the accuracy either. Concerning the explanations supporting network predictions, as presented in Section 3, activation maps from the prototype layer can be upsampled to the size of the input sample to produce a heatmap that identifies the part of the input sample most similar to the learned prototype. It has been shown in [23] that applying upsampling processes to match the size of the input sample can affect the precision of the explanations. Therefore, we keep the feature map dimensions over the network the same as the input sample dimensions (20×2 - detailed in Section 4.1) using fully padded convolutions.

Table 1: Overall architecture of LEXNet.

Input Dims	Operator	Stride	Out Channels	Cum. Params
1×20×2	Conv3x3+BN	1	8	88
8×20×2	LERes Block	1	16	3,088
16×20×2	LERes Block	1	16	7,760
16×20×2	LERes Block	1	32	19,520
32×20×2	LERes Block	1	32	38,080
32×20×2	LProto Layer	-	340	50,880
340×20×2	Max Pooling	-	340	50,880
340×1×1	FC	-	200	118,880

4 EVALUATION SETTING

In this section, we present the methodology employed (dataset, algorithms, hyperparameters and configuration) to evaluate our approach.

4.1 Dataset

Commercial-Grade Dataset Our real-world dataset [17] has been collected from four customer deployments in China. The dataset is composed of the TCP (e.g., HTTP) and UDP (e.g., streaming media, VoIP) traffic activity over four weeks across tens of thousands network devices. Specifically, it contains 9.7M flows/MTS belonging to 200 applications/classes (169 TCP and 31 UDP). For each flow/MTS, we have available some per-packet information (variables: packet size and direction). We set the MTS length to the first 20 packets. This value reflects the relevant time windows to identify the applications and sustain line rate classification of the traffic. Concerning the labeling, each flow has been annotated with application names provided by a commercial-grade deep packet inspection (DPI) engine, i.e., a traditional rule-based method. Traffic encryption in China is not as present as in the Western world yet, so DPI technologies still offer fine-grained view on traffic.

Table 2 presents the structure of the dataset. We observe a class imbalance as the top 10 applications represent more than 40% of the flows. This is characteristic of the traffic classification task and we show in Section 5.1 that our classifier is robust to this class imbalance.

Table 2: Composition of our commercial-grade dataset. Applications are presented in descending order of their popularity.

Applications	Flows (M)	Flows (%)	TCP %
10	4.1	42.9	70.0
20	5.7	59.4	69.5
50	7.8	80.3	74.5
100	9.0	92.9	76.2
200	9.7	100.0	76.6

Other Public Datasets Moreover, to show the generalizability of our results, we have also evaluated LEXNet on the current benchmark dataset for traffic classification (MIRAGE [1] - 100k flows belonging to 40 applications), the recent CESNET-TLS [38] (38M flows belonging to 191 applications) and on the dataset MNIST [34] from computer vision (70k samples belonging to 10 classes).

4.2 Algorithms

We compare our algorithm LEXNet [14] to the state-of-the-art classifiers presented in Section 2. Specifically, we used the authors' implementation of CondenseNetV2 [61], MLSTM-FCN [32], ProtoPNet [10] and XCM [23]. Moreover, we used the PyTorch Hub [46]/ Torchvision [47] implementations of DenseNet [29], EfficientNet [56], GhostNet [26], MobileNetV3 [28], ResNet [27] and ShuffleNetV2 [39]. Then, we have implemented with PyTorch in Python 3 the 1D CNN. Finally, we used the public implementations available for the ensembles (Random Forest [44], XGBoost [11]) and post-hoc explainability methods (Grad-CAM [54], SHAP [37]).

4.3 Hyperparameters

Adopting a conservative approach, we performed a 50% train/50% test split of our dataset while preserving its structure (split provided in [17]). We adopted the same approach for the other traffic datasets (MIRAGE, CESNET-TLS), and kept the train/test split provided for MNIST. Hyperparameters have been set by grid search based on the best average accuracy following a stratified 5-fold cross-validation on the training set. As recommended by the authors, we let the number of LSTM cells varies in {8, 64, 128} for MLSTM-FCN, and the size of the window varies in {20%, 40%, 60%, 80%, 100%} for XCM. Considering the size of our input data (20×2), we let the hyperparameters of ProtoPNet (number and size of the prototypes) vary in {1, 2, 3, 4}. Similarly, we let the size of the prototypes for LEXNet varies in {1, 2, 3, 4}. For the other architectures, we use the default parameters suggested by the authors.

4.4 Configuration

All the models have been trained with 1000 epochs, a batch size of 1024 and the following computing infrastructure: Ubuntu 20.04 operating system, GPU NVIDIA Tesla V100 with 16GB HBM2. With regard to the limited computational resources of network devices, we also evaluate the models without GPU acceleration on an Intel Xeon Platinum 8164 CPU (2.00GHz, 71.5MB L3 Cache).

5 RESULTS AND DISCUSSIONS

In this section, we first present the performance results of LEXNet, then, we discuss the explainability feature of our approach.

5.1 Classification

CNN Backbone First, in order to determine the starting architecture for the backbone of LEXNet, we compare the state-of-the-art classifiers. Table 3 presents the accuracy and inference time of the classifiers. We observe that the model obtaining the best accuracy on our dataset is ResNet (90.4%), while getting the lowest variability across folds (0.1%). This model already allows us to reduce the model size by a factor of five compared to the current state-of-the-art traffic classifiers (1-2M of trainable parameters - see Section 2 - versus 303k for ResNet). Larger or smaller ResNet models do not exhibit a higher level of accuracy (small 138k: 86.7%, large 1M: 89.6% - see left side of Table 3). Then, Table 3 shows all other state-of-the-art classifiers on a comparable model size for a fair comparison: ~ 300k edges - trainable parameters or leafs. ResNet exhibits the best accuracy/inference time trade-off; it obtains the second position with regard to the inference time on both GPU and CPU. Traditional tree-based ensemble methods - Random Forest and XGBoost - obtain an accuracy close to the 1D CNN (84.5%), while being around a thousand times slower. Concerning the state-of-the-art efficient CNNs, they all exhibit a higher inference time than ResNet. In particular, some efficient CNNs with really low FLOPs/Multiply-Adds (e.g., CondenseNetV2: 0.4M) compared to ResNet (2.1M) have a much higher inference time (e.g., CondenseNetV2: GPU 3.1 μ s/CPU 80.3 μ s versus GPU 1.6 μ s/CPU 34.6 μ s). The extensive use of operations that reduce the number of FLOPs (e.g., depthwise and 1x1 convolutions) do not translate into reduction in inference time due to factors like memory access cost [49]. Thus, our experiments show that optimizing FLOPs does not always reflect equivalently into inference

Table 3: Accuracy with standard error and inference time of the state-of-the-art classifiers on the TCP+UDP test set.

Metric	Small	ResNet Medium	Large	1D CNN	Mobile NetV3	Shuffle NetV2	Condense NetV2	Dense Net	MLSTM FCN	XCM	Ghost Net	Efficient Net	RF	XGB
Accuracy (%)	86.7 ± 0.2	90.4 ± 0.1	89.6 ± 0.2	84.5 ± 0.2	85.7 ± 0.3	86.9 ± 0.3	87.9 ± 0.1	89.5 ± 0.4	86.5 ± 0.3	88.1 ± 0.2	88.6 ± 0.3	86.6 ± 0.4	84.4 ± 0.2	84.7 ± 0.2
Inf. GPU (μ s/sample)	0.9	1.6	6.1	1.4	2.2	2.5	3.1	3.3	4.6	8.1	11.5	13.3	-	-
Inf. CPU (μ s/sample)	16.6	34.6	98.6	26.6	48.6	69.8	80.3	91.8	186.3	355	473.8	610.8	1.6E5	6.2E4
# Edges (k)	138	303	1003	274	312	305	326	328	394	315	322	308	349	352
Multi-Adds (M)	0.5	2.1	20.1	0.6	0.7	0.5	0.4	8.3	2.3	2.6	6.9	2.2	-	-

time, and emphasize the interest of comparing model performance on the same model size.

Nonetheless, a model is faster than ResNet: the vanilla 1D CNN (GPU 1.4 μ s/CPU 26.6 μ s versus GPU 1.6 μ s/CPU 34.6 μ s). The absence of residual connection in the 1D CNN can explain this lower inference time. Therefore, we have selected ResNet as starting point for our backbone and worked to reduce its number of weights and inference time by introducing a new residual block (LERes), while maintaining its accuracy as detailed in next section.

LEXNet versus ProtoPNet Our state-of-the-art explainable-by-design CNN baseline is ProtoPNet. Table 4 shows the ablation study from ProtoPNet to LEXNet with our two contributions: a new lightweight and efficient residual block (LERes) and a lightweight prototype layer (LProto). The performances reported correspond to the ones with the best hyperparameter configuration obtained by cross-validation on the training set: prototypes of size (1, 1) for both networks and a fixed number of 2 prototypes per class for ProtoPNet (LEXNet learned 1.7 ± 0.1 prototypes per class on average).

First, ProtoPNet with ResNet as CNN backbone exhibits an accuracy of 86.2% with 199k trainable parameters. This reduction in model size compared to ResNet (303k parameters - see Table 3) comes from the reduction in size of the fully connected network used for classification (ProtoPNet: 400 values as input - 2 similarity scores/prototypes per class). Then, we can see that the replacement of the prototype block of ProtoPNet by LProto significantly reduces the number of parameters (-36% of trainable parameters), while increasing the accuracy of the network by 4% to reach the same accuracy as ResNet (90.4% - see Table 3). As a consequence, the adoption of LProto also leads to a significant reduction in inference time (GPU: -29%/CPU: -24%). Specifically, the reduction of the number of parameters and inference time mainly come from the removal of the convolutional layers, which also decreases the depth of the prototypes. And, the increase in accuracy mainly comes from the L2 regularization (+2.8%) which improves the generalization ability of our approach. Moreover, the introduction of a variable number of prototypes per class implies a reduction of the total number of prototypes compared to the state-of-the-art method (LEXNet 340 - 1.7 ± 0.1 per class - versus ProtoPNet: 400 - 2 prototypes per class), which lowers the number of parameters (-12k) and inference time (GPU: -0.1 μ s/sample/CPU: -11.2 μ s/sample), while maintaining the accuracy (see (4) in Table 4).

Next, the replacement of the original residual block by LERes block allows ResNet to be faster than the vanilla 1D CNN (GPU 1.3 μ s/ CPU 20.3 μ s versus GPU 1.4 μ s/CPU 26.6 μ s), while reducing the number of trainable parameters of its backbone by 19% and maintaining the accuracy (99.3% of the original ResNet). When used

in ProtoPNet, we observe in Table 4 a comparable performance evolution as LERes block reduces the backbone parameters and CPU inference time by 19%, while preserving the accuracy (-0.7%). Our LERes block integrates two new operations (generation of cost-efficient feature maps with a linear transformation, concatenation on shortcut connection), and we observe that both operations reduce the number of trainable parameters and inference time, with the first operation having twice the impact of the second one.

Overall, our explainable-by-design LEXNet is more accurate (+4%), lighter (-40%) and faster (-39%) than the current state-of-the-art explainable-by-design CNN network ProtoPNet, while not inducing a higher variability across folds (0.1%).

Table 4: Ablation study of LEXNet. Blanks mean zero.

	Accuracy (%)	# Params (k)	Inf. GPU (μ s/sample)	Inf. CPU (μ s/sample)
(0) ProtoPNet	86.2 ± 0.1	199	5.5	169.7
(1)-(0)+No Add. Conv	+0.2	-59	-1.5	-29.3
(2)-(1)+Sigmoid	+1.2			-0.6
(3)-(2)+L2 Regularization	+2.8			+1
(4)-(3)+Var. Protos		-12	-0.1	-11.2
(4): (0)+LProto	90.4 ± 0.1	128	3.9	129.6
Summary	+4%	-36%	-29%	-24%
(5)-(4)+Linear	-0.5	-3	-0.1	-11.9
(6)-(5)+Concatenate	-0.2	-6	-0.2	-15
LEXNet: (4)+LERes	89.7 ± 0.1	119	3.6	102.7
Summary	+4%	-40%	-35%	-39%

LEXNet Predictions Our results show that Internet encrypted flows can be classified with a high state-of-the-art accuracy of 89.7% using solely the values of 1.7 ± 0.1 prototypes of size (1, 1) per application based on MTS containing the first 20 packets of a flow, with per-packet size and direction as variables. More particularly, 1.7 prototypes of size (1, 1) per application on average are sufficient to classify both TCP and UDP applications with a high state-of-the-art accuracy of 87.8% for TCP and 95.9% for UDP (consistent with the literature [60]). This best hyperparameter configuration (prototypes of size (1, 1)) also informs us that the information necessary to discriminate applications is often not long sequence of packet sizes or directions, but the combination of both packet sizes and directions at different places of the flow. Then, Internet traffic is highly imbalanced, with a few applications generating most of the flows. As an example, half of the classes (100 classes) represent less than 10% of the total number of flows in our dataset (see Table 2). Our results show that, by maintaining the same overall high accuracy as ResNet and a comparable accuracy per class (see Figure 5), the addition of a prototype layer for explainability doesn't alter the robustness of our model to class imbalance. Among the 100 less popular classes, more than a third of them are classified with an accuracy above 75%.

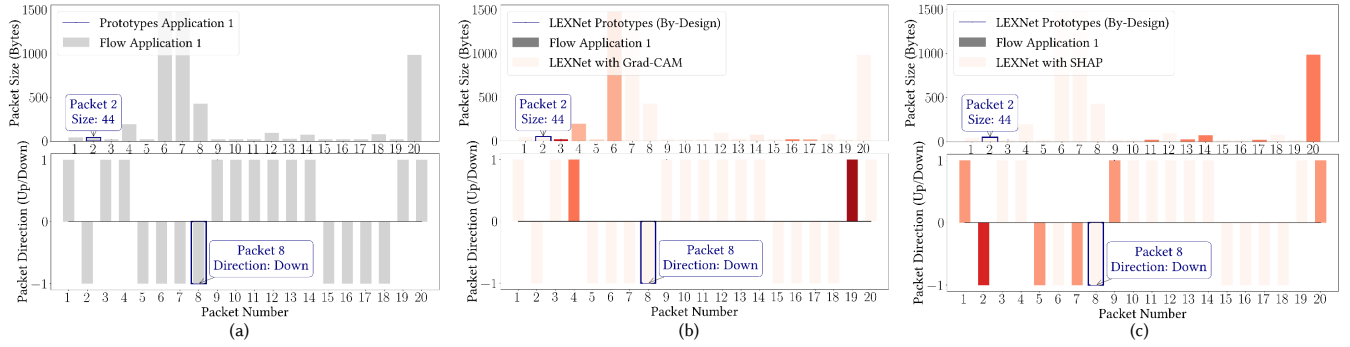


Figure 4: (a) Example of a flow from one TCP application of our dataset with the prototypes identified by LEXNet with its explainability-by-design in blue. (b-c) Same flow with the two prototypes identified by LEXNet with its explainability-by-design in blue versus the post hoc explainability methods Grad-CAM (b) and SHAP (c) in red.

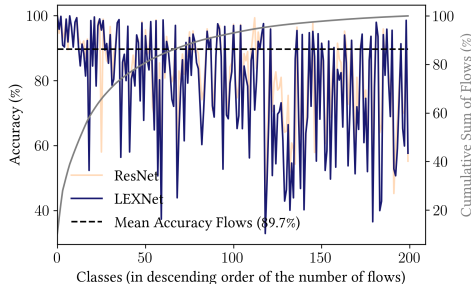


Figure 5: LEXNet and ResNet (with LERes block) accuracy per class on the TCP+UDP test set.

Results on External Datasets To evaluate the generalizability of our results, we have also compared LEXNet to ProtoPNet and ResNet on external datasets (see Table 5). We observe that the results are consistent with the ones obtained on our commercial-grade traffic classification dataset. LEXNet exhibits the same accuracy as ResNet while outperforming the current state-of-the-art explainable-by-design network ProtoPNet (CESNET-TLS: 97.2% versus 94.1%, MIRAGE: 73.6% versus 71.4%, MNIST: 99% versus 97.3%). Moreover, LEXNet significantly reduces the parameters and inference time compared to ProtoPNet (parameters - CESNET-TLS: 111k versus 189k, MIRAGE: 43k versus 81k, MNIST: 39k versus 70k; inference time CPU - CESNET-TLS: 92.1ms versus 146.3ms, MIRAGE: 32.8ms versus 40.9ms, MNIST: 1.1s versus 1.8s).

Table 5: Comparison of LEXNet and the state-of-the-art CNN with explainability-by-design on external datasets.

Dataset	Model	Accuracy (%)	# Params (k)	Inf. GPU (μ s/sample)	Inf. CPU (μ s/sample)
CESNET-TLS	LEXNet	97.2*	111	3.4	92.1
	ProtoPNet	94.1	189	5.1	146.3
MIRAGE	LEXNet	73.6*	43	1.6	32.8
	ProtoPNet	71.4	81	2.3	40.9
MNIST	LEXNet	99.0*	39	16.3	1138
	ProtoPNet	97.3	70	25.6	1834

*Same accuracy as ResNet.

5.2 Explainability

Illustration LEXNet gives as prediction the application associated with the identified class-specific prototypes (1.7 ± 0.1 prototypes on average), and as explanation the corresponding prototypes/regions of size (1, 1) on the input flow/sample as detailed in Section 3. Based on the first 20 packets of a flow from one representative TCP application (having the closest number of prototypes per class to the mean: two), Figure 4a illustrates this explainability by identifying in blue the two class-specific prototypes that have been used for the prediction. The flow is represented with a bar chart for each variable and the two prototypes of size (1, 1) are highlighted by two blue rectangles identifying precisely the region of the input data that has been used for prediction. In this example, a small packet of size 44 in position 2 and a descending packet in position 8 have been detected, which are characteristic of the class containing this TCP application. Another application, e.g., application 2, is characterized by a large packet size of 1480 in position 8 and a descending packet in position 9. When comparing flows from different applications, this representation with prototypes allows the end-user/network expert to identify the signature of each application.

Moreover, we observe that the correlation between the number of prototypes per application of LEXNet (ranging from 1 to 5) and the number of flows per application is negative (-0.24), which means that popular applications can be characterized by a low number of prototypes (e.g., top-20 applications - $\sim 60\%$ of the traffic - have 1 prototype per class). Therefore, in most cases, the explanations provided to the end-user in order to support LEXNet predictions are easily interpretable as they only highlight one location in the flows. Thus, in addition to reducing the number of parameters and inference time as presented in Table 4, the introduction of a variable number of prototypes per class (versus a fixed number of two) provides more relevant explanations by better identifying the discriminative features of each application.

Faithfulness Then, we highlight the faithfulness of LEXNet explanations. We compare the most important regions of size (1, 1) identified by the faithful (by definition) LEXNet explainability-by-design - the class prototypes - to the ones from the state-of-the-art post hoc model-specific method Grad-CAM and model-agnostic method SHAP (methods used in state-of-the-art traffic classifiers -

see section 2). First, based on one sample from one representative TCP application, we show in Figure 4b-c that the post hoc explainability methods, applied on the same model LEXNet, identify none of the expected regions/prototypes used by LEXNet. These methods identify regions (in red) which can be far from the expected ones (e.g., Direction: Grad-CAM #19 versus SHAP #2 versus LEXNet #8). Second, in order to quantitatively assess this difference across the dataset, considering the class-specific prototypes to be identified, we calculate the top-protos (top-1 to top-5 according to the number of prototypes per class) and top-10 accuracy in a similar fashion as the top-1 and top-5 accuracies in computer vision evaluations. The results in Table 6 show that Grad-CAM slightly better identifies the regions of the input data that are used by LEXNet (prototypes) compared to SHAP, but overall both post-hoc methods poorly perform (Grad-CAM: top-protos 8.2%/top-10 38.9%, SHAP top-protos 5.9%/top-10 27.4%). When using the 10 first predicted (1, 1) regions from the post hoc explainability methods, i.e., 25% of the size of the input data, the expected prototypes are identified with an accuracy of less than 40%. Therefore, this experiment clearly emphasizes the importance of adopting explainable-by-design methods compared to post-hoc ones in order to ensure faithfulness.

Table 6: Faithfulness evaluation on the TCP+UDP train set: ability of the state-of-the-art post hoc explainability methods to identify the prototypes used by LEXNet to predict.

LEXNet	By-Design	+ Grad-CAM	+ SHAP
Top-Protos Regions Accuracy (%)	100.0	8.2	5.9
Top-10 Regions Accuracy (%)	-	38.9	27.4

The Cost of Explainability Table 7 shows the performance of LEXNet (accuracy/size/inference) in comparison with ResNet, which can only rely on state-of-the-art post hoc explainability methods, and the current state-of-the-art explainable-by-design CNN ProtoPNet with the same backbone as LEXNet. In addition to the faithfulness, we observe that LEXNet is around 2.5× faster than ResNet with LERes block using the most popular post-hoc explainability method for CNNs (the saliency method Grad-CAM), while maintaining the accuracy (89.7%) and halving the model size (119k parameters). Thus, the lightweight design of LEXNet meets the need for fast inference that ensures line rate classification on network devices with limited computational resources (~10k classifications/s on CPUs, versus ProtoPNet: 6k classifications/s and ResNet with Grad-CAM: 4k classifications/s), which is crucial for latency sensitive traffic.

Table 7: Comparison of LEXNet and the state-of-the-art CNNs with explainability-by-design and post-hoc explainability on the TCP+UDP test set.

	Accuracy (%)	# Params (k)	Inf. GPU (μ s/sample)	Inf. CPU (μ s/sample)
ResNet* + Grad-CAM	89.7	294	9.5	278.6
ResNet* + SHAP	89.7	294	8.3E3	6.8E4
ProtoPNet*	86.2	199	5.2	142.8
LEXNet*	89.7	119	3.6	102.7

*All networks adopt LERes block to keep the same basis of comparison.

Nonetheless, this combination obtained does not come at no cost: the explainability-by-design of LEXNet has a cost on the inference

time compared to the best performing state-of-the-art CNN ResNet without explainability methods (GPU 3.6 μ s/CPU 102.7 μ s versus GPU 1.3 μ s/CPU 20.3 μ s) and remains an open challenge. To the best of our knowledge, our study is the first to quantify the impact of explainability on the prediction performance, as well as the size and inference time of a model. Based on our experiments, around 80% of this additional inference time is due to the L2 distance calculations in the prototype layer to generate the similarity matrices – which thus could benefit some optimization.

6 CONCLUSION

We have presented LEXNet, a new lightweight, efficient and explainable-by-design CNN for traffic classification which relies on a new residual block and prototype layer. LEXNet exhibits a significantly lower model size and inference time compared to the state-of-the-art explainable-by-design CNN ProtoPNet, while being more accurate. Plus, LEXNet is also lighter and faster than the best performing state-of-the-art neural network ResNet with post hoc explainability methods, while maintaining its accuracy. Our results show that Internet encrypted flows can be classified with a high state-of-the-art accuracy using solely the values of less than two prototypes of size (1, 1) on average per application based on MTS containing the first 20 packets of a flow, with per-packet size and direction as variables. These class prototypes detected on a flow can be given to the end-user as a simple and faithful explanation to support LEXNet application prediction.

LEXNet has been developed in close collaboration with our Huawei business product division. Therefore, the performance and explainability of LEXNet meet the requirements for deployment: both for hardware computational resources (e.g., ARM CPU specifications typically used in routers - 10k classifications/second), and for the format of explanations needed by network experts (prototypes). Concerning the usage of LEXNet in production, in addition to traditional solutions for model monitoring (e.g., data drift detection, concept drift identification), LEXNet explainability can be leveraged as a supplementary way to track model drift by assessing the consistency across time of the model prediction performance and the similarity to the prototypes used to support the predictions.

In our future work, we would like to minimize the impact of the explainability feature on the inference time of LEXNet by further optimizing the generation of the similarity matrices in the prototype layer. It would also be interesting to investigate applications in collaboration with domain experts where LEXNet would be beneficial (e.g., mobility [31], natural disasters early warning [22], smart farming [25]), and to further augment the capabilities of LEXNet with such domain requirements.

REFERENCES

- [1] G. Aceto, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapè. 2019. MIRAGE: Mobile-app Traffic Capture and Ground-truth Creation. In *Proceedings of the IEEE 4th International Conference on Computing, Communication and Security*.
- [2] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapè. 2019. MIMETIC: Mobile Encrypted Traffic Classification using Multimodal Deep Learning. *Computer Networks* 165 (2019), 106944.
- [3] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapè. 2019. Mobile Encrypted Traffic Classification Using Deep Learning: Experimental Evaluation, Lessons Learned, and Challenges. *IEEE Transactions on Network and Service Management* 16, 2 (2019), 445–458.

- [4] G. Aceto, D. Ciunzio, A. Montieri, and A. Pescapè. 2020. Toward Effective Mobile Encrypted Traffic Classification through Deep Learning. *Neurocomputing* 409 (2020), 306–315.
- [5] A. Bagnall, J. Lines, and E. Keogh. 2018. The UEA UCR Time Series Classification Archive. (2018).
- [6] M. Baydogan and G. Runger. 2016. Time Series Representation and Similarity Based on Local Autopatterns. *Data Mining and Knowledge Discovery* 30, 2 (2016), 476–509.
- [7] C. Beliard, A. Finamore, and D. Rossi. 2020. Opening the Deep Pandora Box: Explainable Traffic Classification. In *Proceedings of the IEEE Conference on Computer Communications Workshops*.
- [8] R. Boutaba, M. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, and O. Caccido. 2018. A Comprehensive Survey on Machine Learning for Networking: Evolution, Applications and Research Opportunities. *Journal of Internet Services and Applications* 9, 16 (2018).
- [9] L. Breiman. 2001. Random Forests. *Machine Learning* 45 (2001), 5–32.
- [10] C. Chen, O. Li, D. Tao, A. Barnett, C. Rudin, and J. Su. 2019. This Looks Like That: Deep Learning for Interpretable Image Recognition. In *Proceedings of the 33rd Conference on Neural Information Processing Systems*.
- [11] T. Chen and C. Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- [12] Z. Chen, Y. Bei, and C. Rudin. 2020. Concept Whitening for Interpretable Image Recognition. *Nature Machine Intelligence* 2 (2020), 772–782.
- [13] Cisco. 2018. Application Visibility and Control User Guide. *Online* (2018). https://www.cisco.com/c/en/us/td/docs/ios/solutions_docs/avc/guide/avc-user-guide.html
- [14] Code. 2023. LEXNet. *Online* (2023). <https://github.com/XAISeries/LEXNet>
- [15] M. Crotti, M. Dusi, F. Gringoli, and L. Salgarelli. 2007. Traffic Classification through Simple Statistical Fingerprinting. *ACM SIGCOMM Computer Communication Review* 37, 1 (2007), 5–16.
- [16] A. Dainotti, A. Pescapè, and K. Claffy. 2012. Issues and Future Directions in Traffic Classification. *IEEE Network* 26, 1 (2012), 35–40.
- [17] Dataset. 2023. AppClassNet. *Online* (2023). https://figshare.com/articles/dataset/AppClassNet_-_A_commercial-grade_dataset_for_application_identification_research/20375580
- [18] K. Dias, M. Pongelupé, W. Caminhas, and L. de Errico. 2019. An Innovative Approach for Real-Time Network Traffic Classification. *Computer Networks* 158 (2019), 143–157.
- [19] V. Dignum. 2017. Responsible Artificial Intelligence: Designing AI for Human Values. *ITU Journal* 1 (2017), 1–8.
- [20] M. Du, N. Liu, and X. Hu. 2020. Techniques for Interpretable Machine Learning. *Commun. ACM* (2020).
- [21] G. Elsayed, S. Kornblith, and Q. Le. 2019. Saccader: Improving Accuracy of Hard Attention Models for Vision. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*.
- [22] K. Fauvel, D. Balouek-Thomert, D. Melgar, P. Silva, A. Simonet, G. Antoniu, A. Costan, V. Masson, M. Parashar, I. Roderio, and A. Termier. 2020. A Distributed Multi-Sensor Machine Learning Approach to Earthquake Early Warning. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence*.
- [23] K. Fauvel, T. Lin, V. Masson, E. Fromont, and A. Termier. 2021. XCM: An Explainable Convolutional Neural Network for Multivariate Time Series Classification. *Mathematics* 9, 23 (2021).
- [24] K. Fauvel, V. Masson, and E. Fromont. 2020. A Performance-Explainability Framework to Benchmark Machine Learning Methods: Application to Multivariate Time Series Classifiers. In *Proceedings of the IJCAI-PRICAI Workshop on Explainable Artificial Intelligence*.
- [25] K. Fauvel, V. Masson, É. Fromont, P. Faverdin, and A. Termier. 2019. Towards Sustainable Dairy Management - A Machine Learning Enhanced Method for Estrus Detection. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*.
- [26] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu. 2020. GhostNet: More Features from Cheap Operations. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.
- [27] K. He, X. Zhang, S. Ren, and J. Sun. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.
- [28] A. Howard, M. Sandler, B. Chen, W. Wang, L. Chen, M. Tan, G. Chu, V. Vasudevan, Y. Zhu, R. Pang, H. Adam, and Q. Le. 2019. Searching for MobileNetV3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*.
- [29] G. Huang, Z. Liu, L. Van Der Maaten, and K. Weinberger. 2017. Densely Connected Convolutional Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.
- [30] Huawei. 2021. Service Awareness. *Online* (2021). <https://e.huawei.com/ph/material/MaterialList?id={BEB24273-A3D3-45FD-BCE3-C696EC272720}>
- [31] R. Jiang, X. Song, D. Huang, X. Song, T. Xia, Z. Cai, Z. Wang, K. Kim, and R. Shibasaki. 2019. DeepUrbanEvent: A System for Predicting Citywide Crowd Dynamics at Big Events. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*.
- [32] F. Karim, S. Majumdar, H. Darabi, and S. Harford. 2019. Multivariate LSTM-FCNs for Time Series Classification. *Neural Networks* 116 (2019), 237–245.
- [33] I. Karlsson, P. Papapetrou, and H. Boström. 2016. Generalized Random Shapelet Forests. *Data Mining and Knowledge Discovery* 30, 5 (2016), 1053–1085.
- [34] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. 1998. Gradient-Based Learning Applied to Document Recognition. *Proc. IEEE* (1998).
- [35] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li. 2019. FS-Net: A Flow Sequence Network for Encrypted Traffic Classification. In *Proceedings of the IEEE Conference on Computer Communications*. 1171–1179.
- [36] M. Lotfollahi, M. Siavoshani, R. Zade, and M. Saberian. 2020. Deep Packet: A Novel Approach for Encrypted Traffic Classification Using Deep Learning. *Soft Computing* 24, 3 (2020), 1999–2012.
- [37] S. Lundberg and S. Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*.
- [38] J. Luxemburk and T. Čejka. 2023. Fine-grained TLS services classification with reject option. *Computer Networks* 220 (2023), 109467.
- [39] N. Ma, X. Zhang, H. Zheng, and J. Sun. 2018. ShuffleNetV2: Practical Guidelines for Efficient CNN Architecture Design. In *Proceedings of the 14th European Conference on Computer Vision*.
- [40] A. Nascita, A. Montieri, G. Aceto, D. Ciunzio, V. Persico, and A. Pescapè. 2021. XAI Meets Mobile Traffic Classification: Understanding and Improving Multimodal Deep Learning Architectures. *IEEE Transactions on Network and Service Management* 18, 4 (2021), 4225–4246.
- [41] ntop. 2014. nDPI. *Online* (2014). <https://www.ntop.org/products/deep-packet-inspection/ndpi/>
- [42] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar. 2018. Towards the Deployment of Machine Learning Solutions in Network Traffic Classification: A Systematic Survey. *IEEE Communications Surveys & Tutorials* 21, 2 (2018), 1988–2014.
- [43] European Parliament and Council. 2021. Artificial Intelligence Act. *European Union Law* (2021).
- [44] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [45] P. Phillips, C. Hahn, P. Fontana, A. Yates, K. Greene, D. Broniatowski, and M. Przybicki. 2021. Four Principles of Explainable Artificial Intelligence. *US National Institute of Standards and Technology* (2021).
- [46] PyTorch. 2021. PyTorch Hub. *Online* (2021). <https://pytorch.org/hub/research-models>
- [47] PyTorch. 2021. Torchvision v0.11.0. *Online* (2021). <https://pytorch.org/vision/0.11/>
- [48] S. Rezaei, B. Kroencke, and X. Liu. 2020. Large-Scale Mobile App Identification Using Deep Learning. *IEEE Access* 8 (2020), 348–362.
- [49] T. Ridnik, H. Lawen, A. Noy, E. Ben, B. Sharir, and I. Friedman. 2021. TRResNet: High Performance GPU-Dedicated Architecture. In *Proceedings of the IEEE Winter Conference on Applications of Computer Vision*.
- [50] Rohde&Schwarz. 2020. R&S® PACE 2. *Online* (2020). <https://www.ipoque.com/products/dpi-engine-rs-pace-2-for-application-awareness>
- [51] C. Rudin. 2019. Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead. *Nature Machine Intelligence* 1 (2019), 206–215.
- [52] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. Chen. 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.
- [53] P. Schäfer and U. Leser. 2017. Multivariate Time Series Classification with WEASEL + MUSE. *ArXiv* (2017).
- [54] R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra. 2017. Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. In *Proceedings of the IEEE International Conference on Computer Vision*.
- [55] S. Seto, W. Zhang, and Y. Zhou. 2015. Multivariate Time Series Classification Using Dynamic Time Warping Template Selection for Human Activity Recognition. In *Proceedings of the IEEE Symposium Series on Computational Intelligence*.
- [56] M. Tan and Q. Le. 2019. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *Proceedings of the 36th International Conference on Machine Learning*.
- [57] K. Tuncel and M. Baydogan. 2018. Autoregressive Forests for Multivariate Time Series Modeling. *Pattern Recognition* 73 (2018), 202–215.
- [58] S. Valenti, D. Rossi, A. Dainotti, A. Pescapè, A. Finamore, and M. Mellia. 2013. *Reviewing Traffic Classification*. Springer Berlin Heidelberg, 123–147.
- [59] X. Wang, S. Chen, and J. Su. 2020. Automatic Mobile App Identification From Encrypted Traffic With Hybrid Neural Networks. *IEEE Access* 8 (2020), 182065–182077.

- [60] L. Yang, F. Alessandro, F. Jun, and D. Rossi. 2021. Deep Learning and Zero-Day Traffic Classification: Lessons Learned from a Commercial-Grade Dataset. *IEEE Transactions on Network and Service Management* 18, 4 (2021), 4103–4118.
- [61] L. Yang, H. Jiang, R. Cai, Y. Wang, S. Song, G. Huang, and Q. Tian. 2021. CondenseNetV2: Sparse Feature Reactivation for Deep Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.
- [62] S. Zagoruyko and N. Komodakis. 2016. Wide Residual Networks. In *Proceedings of the British Machine Vision Conference*.
- [63] Q. Zhang, Y. Wu, and S. Zhu. 2018. Interpretable Convolutional Neural Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.
- [64] X. Zhang, Y. Gao, J. Lin, and C. Lu. 2020. TapNet: Multivariate Time Series Classification with Attentional Prototypical Network. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence*.